HDOC Day-72

Student Attendance Management Using Dynamic Memory Allocation in C

Problem Statement

Develop a C program to manage attendance in a dynamically resizable matrix. Each row represents a student, each column a working day, 1 represents present and 0 represents absent. Use pointers, malloc(), realloc(), separate functions, input validation, attendance percentages, and free() to release memory.

Algorithm

1. Start and read a positive initial student count and working-day count.

2. Allocate one contiguous attendance matrix using malloc() and check allocation.

3. Read attendance (only 0 or 1) for each student and day using pointer arithmetic.

4. Display a menu for adding students, viewing records, calculating percentages, or exiting.

5. To add students, validate the count and use realloc() with a temporary pointer; preserve original records and enter the new rows.

6. Display attendance row by row using pointer notation.

7. For each student, count present days and calculate (present / working days) x 100.

8. Repeat until Exit; free the allocated matrix and stop.

Test Case Table

TC	Inputs / action	Expected result
1	3 students, 3 days; rows 1 0 0 / 1 0 1 / 1 1 1; choose 2	Displays three original attendance rows.
2	Same initial data; choose 3	33.3%, 66.7%, 100.0%.
3	Same initial data; add 3 students: 1 0 0 / 1 1 1 / 1 0 1; choose 2	Displays all six rows, preserving first three.
4	After adding 3 as in TC3; choose 3	33.3%, 66.7%, 100.0%, 33.3%, 100.0%, 66.7%.
5	1 student, 1 day; value 1; choose 3	100.0%.
6	1 student, 2 days; values 0 0; choose 3	0.0%.
7	Initial student count 0, then 2; 2 days; rows 1 0 / 0 1; choose 3	Rejects 0 and accepts 2; both students 50.0%.
8	1 student, 2 days; enter 2, then 1 for first day; 0 for second; choose 2	Rejects attendance 2, accepts 1; displays 1 0.
9	1 student, 1 day; value 1; choose 1, add 1 student with value 0; choose 3	First student 100.0%; second 0.0%.
10	1 student, 1 day; value 1; choose 4	Exits and releases allocated memory.

Note: Each test case is an independent program run unless otherwise stated. For the initial three-student example, enter the three rows before choosing a menu option.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int read_int(const char *prompt, int low, int high) {
    int value;
    for (;;) {
        printf("%s", prompt);
        if (scanf("%d", &value) != 1) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {}
            printf("Invalid input. Enter an integer.\n");
            if (ch == EOF) exit(1);
        } else if (value < low || value > high) {
            printf("Enter a value from %d to %d.\n", low, high);
        } else return value;
    }
}

int *create_matrix(int students, int days) {
    int *a = malloc((size_t)students * (size_t)days * sizeof *a);
    if (!a) { printf("Memory allocation failed.\n"); exit(1); }
    return a;
}

void record_attendance(int *a, int first, int students, int days) {
    int i, j;
    for (i = first; i < students; ++i) {
        printf("Student %d:\n", i + 1);
        for (j = 0; j < days; ++j) {
            char prompt[80];
            snprintf(prompt, sizeof prompt, "  Day %d (0=Absent, 1=Present): ", j + 1);
            *(a + i * days + j) = read_int(prompt, 0, 1);
        }
    }
}

int *add_students(int *a, int *students, int days, int count) {
    int old = *students;
    int new_count = old + count;
    int *tmp;
    if (count <= 0 || new_count > 1000) {
        printf("Invalid number of additional students.\n");
        return a;
    }
    tmp = realloc(a, (size_t)new_count * (size_t)days * sizeof *a);
    if (!tmp) { printf("Memory reallocation failed; original data preserved.\n"); return a; }
    *students = new_count;
    record_attendance(tmp, old, new_count, days);
    return tmp;
}

void display_records(const int *a, int students, int days) {
    int i, j;
    printf("\nAttendance matrix (1=Present, 0=Absent):\n");
    for (i = 0; i < students; ++i) {
        printf("Student %d: ", i + 1);
        for (j = 0; j < days; ++j) printf("%d ", *(a + i * days + j));
        printf("\n");
    }
}

void attendance_percentages(const int *a, int students, int days) {
    int i, j, present;
    printf("\nAttendance percentages:\n");
    for (i = 0; i < students; ++i) {
        present = 0;
        for (j = 0; j < days; ++j) present += *(a + i * days + j);
        printf("Student %d: %.1f%%\n", i + 1, 100.0 * present / days);
```

```
    }
}

void release_matrix(int *a) { free(a); }

int main(void) {
    int students = read_int("Enter initial number of students (1-1000): ", 1, 1000);
    int days = read_int("Enter working days (1-366): ", 1, 366);
    int *attendance = create_matrix(students, days);
    int choice;
    record_attendance(attendance, 0, students, days);
    do {
        printf("\n--- ATTENDANCE MENU ---\n");
        printf("1. Add students\n2. Display attendance\n3. Attendance percentages\n4. Exit\n");
        choice = read_int("Enter choice: ", 1, 4);
        switch (choice) {
            case 1: {
                int count = read_int("How many students to add (1-1000)? ", 1, 1000);
                attendance = add_students(attendance, &students, days, count);
                break;
            }
            case 2: display_records(attendance, students, days); break;
            case 3: attendance_percentages(attendance, students, days); break;
            case 4: printf("Exiting program. Memory released.\n"); break;
        }
    } while (choice != 4);
    release_matrix(attendance);
    return 0;
}
```

Compilation and Execution

Save as hdoc72.c. Compile: gcc hdoc72.c -o hdoc72 Run: ./hdoc72

Compilation and Execution



Test Case 1

Input: 3 students, 3 days; rows 1 0 0 / 1 0 1 / 1 1 1; choose 2

Expected: Displays three original attendance rows.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc72
Enter initial number of students (1-1000): 3
Enter working days (1-366): 3
Student 1:
Day 1 (0=Absent, 1=Present): 1 0 0
Day 2 (0=Absent, 1=Present): Day 3 (0=Absent, 1=Present): Student 2:
Day 1 (0=Absent, 1=Present): 1 0 1
Day 2 (0=Absent, 1=Present): Day 3 (0=Absent, 1=Present): Student 3:
Day 1 (0=Absent, 1=Present): 1 1 1
Day 2 (0=Absent, 1=Present): Day 3 (0=Absent, 1=Present):
── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 2

Attendance Matrix:
Student 1: 1 0 0
Student 2: 1 0 1
Student 3: 1 1 1
```

Test Case 2

Input: Same initial data; choose 3

Expected: 33.3%, 66.7%, 100.0%.

```
── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 3

Attendance Percentages:
Student 1: 33.3%
Student 2: 66.7%
Student 3: 100.0%
```

Test Case 3

Input: Same initial data; add 3 students: 1 0 0 / 1 1 1 / 1 0 1; choose 2

Expected: Displays all six rows, preserving first three.

```
── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 1
How many students to add (1-1000)? 3
Student 4:
Day 1 (0=Absent, 1=Present): 1 0 0
Day 2 (0=Absent, 1=Present): Day 3 (0=Absent, 1=Present): Student 5:
Day 1 (0=Absent, 1=Present): 1 1 1
Day 2 (0=Absent, 1=Present): Day 3 (0=Absent, 1=Present): Student 6:
Day 1 (0=Absent, 1=Present): 1 0 1
Day 2 (0=Absent, 1=Present): Day 3 (0=Absent, 1=Present):
── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 2

Attendance Matrix:
Student 1: 1 0 0
Student 2: 1 0 1
Student 3: 1 1 1
Student 4: 1 0 0
Student 5: 1 1 1
Student 6: 1 0 1
```

Test Case 4

Input: After adding 3 as in TC3; choose 3

Expected: 33.3%, 66.7%, 100.0%, 33.3%, 100.0%, 66.7%.

```
── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 3

Attendance Percentages:
Student 1: 33.3%
Student 2: 66.7%
Student 3: 100.0%
Student 4: 33.3%
Student 5: 100.0%
Student 6: 66.7%
```

Test Case 5

Input: 1 student, 1 day; value 1; choose 3

Expected: 100.0%.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc72
Enter initial number of students (1-1000): 1
Enter working days (1-366): 1
Student 1:
Day 1 (0=Absent, 1=Present): 1

── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 3

Attendance Percentages:
Student 1: 100.0%
```

Test Case 6

Input: 1 student, 2 days; values 0 0; choose 3
Expected: 0.0%.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc72
Enter initial number of students (1-1000): 1
Enter working days (1-366): 2
Student 1:
Day 1 (0=Absent, 1=Present): 0 0
Day 2 (0=Absent, 1=Present):
── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 3

Attendance Percentages:
Student 1: 0.0%
```

Test Case 7

Input: Initial student count 0, then 2; 2 days; rows 1 0 / 0 1; choose 3
Expected: Rejects 0 and accepts 2; both students 50.0%.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc72
Enter initial number of students (1-1000): 0
Enter a value from 1 to 1000.
Enter initial number of students (1-1000): 2
Enter working days (1-366): 2
Student 1:
Day 1 (0=Absent, 1=Present): 1 0
Day 2 (0=Absent, 1=Present): Student 2:
Day 1 (0=Absent, 1=Present): 0 1
Day 2 (0=Absent, 1=Present):
── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 3

Attendance Percentages:
Student 1: 50.0%
Student 2: 50.0%
```

Test Case 8

Input: 1 student, 2 days; enter 2, then 1 for first day; 0 for second; choose 2
Expected: Rejects attendance 2, accepts 1; displays 1 0.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc72
Enter initial number of students (1-1000): 1
Enter working days (1-366): 2
Student 1:
Day 1 (0=Absent, 1=Present): 2
Enter a value from 0 to 1.
Day 1 (0=Absent, 1=Present): 1
Day 2 (0=Absent, 1=Present): 0

── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 2

Attendance Matrix:
Student 1: 1 0
```

Test Case 9

Input: 1 student, 1 day; value 1; choose 1, add 1 student with value 0; choose 3
Expected: First student 100.0%; second 0.0%.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc72
Enter initial number of students (1-1000): 1
Enter working days (1-366): 1
Student 1:
Day 1 (0=Absent, 1=Present): 1

── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 1
How many students to add (1-1000)? 1
Student 2:
Day 1 (0=Absent, 1=Present): 0

── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 3

Attendance Percentages:
Student 1: 100.0%
Student 2: 0.0%
```

Test Case 10

Input: 1 student, 1 day; value 1; choose 4

Expected: Exits and releases allocated memory.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc72
Enter initial number of students (1-1000): 1
Enter working days (1-366): 1
Student 1:
Day 1 (0=Absent, 1=Present): 1

── ATTENDANCE MENU ──
1. Add students
2. Display attendance
3. Attendance percentages
4. Exit
Enter choice: 4
Exiting program. Memory released.
```